Java Programming Lab Record

Vignana Bharathi Institute of Technology

Experiment 1

Use Eclipse and NetBeans platform and acquaint yourself with various means. Create a test project, add a test class, and run it. See how you can use auto suggestions, autofill. Try code formatter and code refactoring like renaming variables, methods, and classes. Try debug step by step with a small program of about 10 to 15 lines which contains at least one if else condition and for loop.

ForDemo.java

```
import java.util.Scanner
class ForDemo
{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in)
        System.out.println("Enter the number:");
        int n=sc.nextInt();
        int t=n;
        for(int sum=0,r;n!=0;n=n/10)
        {
            r=n%10
            sum=sum+r*r*r;
        }
        if(sum==t)
        {
            System.out.println("Armstrong");
        }
        else
        {
            System.out.println("Not a Armstrong");
        }
    }
}
```

Output:

```
Enter the number:
153
Armstrong
```

Experiment 2

Write a java program to demonstrate mutability of StringBuffer class to find out whether a given string is palindrome or not.

StringBufferDemo.java

```
package logicBuilding;
import java.io.*;
public class StringBufferDemo
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter the String:");
        String str1=br.readLine();
        StringBuffer sb=new StringBuffer(str1);
        sb.reverse();
        String str2=String.valueOf(sb);
        System.out.println(str2);
        if(str1.equals(str2))
            System.out.println("Palindrome");
        else
            System.out.println("Not a Palindrome");
    }
```

```
}
```

Output 1:

```
Enter the String:
272
Palindrome
```

Output 2:

```
Enter the String:
dad
Palindrome
```

Experiment 3

Write a java program to demonstrate Method Overloading.

MethodOverloadingDemo.java

```
class Person
{
    private String name;
    private int age;
    void setInput()
    {
        name="Arjun";
        age=20;
    }
    void setInput(String nam)
    {
        name=nam;
        age=25;
    }
    void setInput(String nam,int ag)
    {
        name=nam;
        age=ag;
    }
    public void talk()
    {
        System.out.println("Name="+name);
        System.out.println("age="+age);
    }
}
public class MethodOverloadingDemo
{
    public static void main(String args[])
    {
        System.out.println("Welcome to Java");
        Person p1=new Person();
        p1.setInput();
        p1.talk();
        Person p2=new Person();
        p2.setInput("Raju");
        p2.talk();
        Person p3=new Person();
        p3.setInput("Rani",22);
        p3.talk();
    }
}
```

Output:

```
Welcome to Java
Name=Arjun
age=20
Name=Raju
age=25
Name=Rani
age=22
```

Experiment 4

Write a java program to demonstrate Constructor Overloading.

ConstructorOverloadingDemo.java

```
class Person
{
    private String name;
    private int age;
    Person()
    {
        name="Surya"
        age=18;
    }
    Person(String nam)
    {
        name=nam;
        age=25;
    }
    Person(String nam,int ag)
    {
        name=nam;
        age=ag;
    }
    public void talk()
    {
        System.out.println("Name="+name);
        System.out.println("Age=",age);
    }
}
public class ConstructorOverloadingDemo
{
    public static void main(String args[])
    {
        System.out.println("Welcome to Java");
        Person p1=new Person();
        p1.talk();
        Person p2=new Person("Raju");
        p2.talk();
        Person p3=new Person("Rani",22);
        p3.talk();
    }
}
```

Output:

```
Welcome to Java
Name=Surya
Age=18
Name=Raju
Age=25
Name=Rani
Age=22
```

Experiment 5

Write a java program to demonstrate Method Overloading.

Polymorphism1.java

```
package Inherit;
class Parent10
{
    void marry()
    {
        System.out.println("Black");
    }
}
class Child10 extends Parent10
{
```

```
    void marry()
    {
        System.out.println("Red");
    }
}
class Polymorphism1
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
        Child10 c=new Child10();
        c.marry()
    }
}
```

Output:

```
Hello world
Red
```

Experiment 6

Write a java program using 'this' and 'super' keywords.

Inheritance3.java

```
class Parent11
{
    Parent11()
    {
        System.out.println("Parent class Zero Argument Constructor");
    }
}
class Child11 extends Parent11
{
    Child11()
    {
        this(10);
        System.out.println("Child class Zero Argument Constructor");
    }
    Child11(int a)
    {
        super();
        System.out.println("Child class One Argument Constructor");
    }
}
class Inheritance3 {
    public static void main(String[] args) {
        System.out.println("Hello world!!")
        new Child11();
    }
}
```

Output:

```
HelloWorld
Parent class Zero Argument Constructor
Child Class One Argument Constructor
Child Class Zero Argument Constructor
```

Experiment 7

7a) Write a java program to create a user defined package named "Sample" and demonstrate importing this package in other program.

UsingPackages.java

```
package pack;
public class Additionpack
{
    int a,b;
```

```
    public Additionpack(int x,int y)
    {
        a=x;
        b=y;
    }
    public void sum()
    {
        System.out.println("sum="+(a+b));
    }
}
package pack;
public class Substractionpack
{
    private double d1,d2;
    public Substractionpack(double d,double e)
    {
        d1=d;
        d2=e;
    }
    public void sub()
    {
        System.out.println("Sub="+(d1-d2));
    }
}
package Temp;
import pack.Additionpack;
import pack.Substractionpack;
class UsingPackages
{
    public static void main(String args[])
    {
        Additionpack obj1=new Additionpack(50,50);
        obj1.sum()
        Substractionpack obj2=new Substractionpack(10.5,45.5);
        obj2.sub();
    }
}
```

Output:

```
Sum=100
Sub=-35.0
```

7b) Write a java program to demonstrate multiple inheritance by using interface.

MultipleInterface.java

```
interface MyInterface3
{
    void greetings();
}
interface MyInterface4
{
    void greetings();
}
class MyClass4 implements MyInterface3,MyInterface4
{
    public void greetings()
    {
        System.out.println("Multiple inheritance using Interface");
    }
}
public class MultipleInterface
{
    public static void main(String[] args)
    {
        MyClass4 obj1=new MyClass4();
        obj1.greetings();
    }
}
```

Output:

```
Multiple inheritance using Interface
```

Experiment 8

Write a java program to demonstrate Abstract Class.

AbstractClasses.java

```
package ABSTRACT;
abstract class MyClass
{
    abstract void calculate(double x);
}
class Sub1 extends MyClass
{
    void calculate(double x)
    {
        System.out.println("Square="+(x*x));
    }
}
class Sub2 extends MyClass
{
    void calculate(double x)
    {
        System.out.println("Square root="+Math.sqrt(x));
    }
}
class Sub3 extends MyClass
{
    void calculate(double x)
    {
        System.out.println("Cube="+Math.pow(x,3));
    }
}
class AbstractClasses
{
    public static void main(String args[])
    {
        Sub1 obj1=new Sub1();
        Sub2 obj2=new Sub2();
        Sub3 obj3=new Sub3();
        obj1.calculate(3);
        obj2.calculate(4);
        obj3.calculate(5);
    }
}
```

Output:

```
Square=9.0
Square root=2.0
Cube=125.0
```

Experiment 9

Write a java program that implements a multi-thread application that has 3 threads. 1st thread generates a random integer every second and if the value is even then 2nd thread computes the square of number and prints. If the value is odd the 3rd thread will print the value of cube of the number.

RandomTest.java

```
import java.util.*;
class Rthread extends Thread
{
    Random r=new Random();
    for(int i=0;i<10;i++)
    {
        int k=r.nextInt(20);
```

```
        System.out.println(k);
        s.start();
    }
    else
    {
        cbthread c=new cbthread();
        c.start();
        {
        try
        {
            Thread.sleep(1000);
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
class Sqthread extends Thread
{
    int n;
    sqthread(int num)
    {
        n=num;
    }
    public void run()
    {
        System.out.println(n*n);
    }
}
class cbthread extends Thread
{
    int n;
    cbthread(int num)
    {
        n=num;
    }
    public void run()
    {
        System.out.println(n*n*n)
    }
}
class RandomTest
{
    public static void main(String args[])
    {
        Rthread rt=new Rthread();
        rt.start();
    }
}
```

Output:

```
9
729
4
16
5
125
3
27
10
100
0
0
```

Experiment 10

Write a Java program for the following: Create a doubly linked list of elements. Delete a given element from the above list. Display the contents of the list after deletion.

DoublyLinkedListDeletion.java

```
class DoublyLinkedList
{
    class Node
    {
        int data;
        Node prev,next;
        Node(int data)
        {
            this.data=data;
            this.prev=null;
            this.next=null;
        }
    }
    private Node head;
    public void insert(int data)
    {
        Node newNode=new Node(data);
        if(head==null)
        {
            head=newNode;
            return;
        }
        Node temp=head;
        while(temp.next!=null)
        {
            temp=temp.next()
        }
        temp.next=newNode;
        newNode.prev=temp;
    }
    public void delete(int key)
    {
        Node temp=head;
        while(temp!=null;temp.data!=key)
        {
            temp=temp.next;
        }
        if(temp==null)
        {
            head=temp.next;
        }
        if(temp.next!=null)
        {
            temp.prev.next=temp.next;
        }
        System.out.println("Element"+key+"deleted
                           successfully");
    }
    public void Display()
    {
        node temp=head
        if(temp==null)
        {
            System.out.println("List is empty")
            return;
        }
        System.out.println("List is empty")
        return;
    }
    System.out.println("Doubly linked list");
    while(temp!=null)
    {
        System.out.println(temp.data+" ");
        temp=temp.next;
    }
    System.out.println();
}
public class DoublyLinkedListDeletion
{
    public static void main(String args[])
    {
```

```
        DoublyLinkedList List=new DoublyLinkedList();
        List.insert(10);
        List.insert(20);
        List.insert(30);
        List.insert(40);
        List.insert(50);
        System.out.println("Before Deletion:");
        List.display();
        List.Deletion(30);
        System.out.println("After Deletion:");
        List.display();
    }
}
```

Output:

```
Before Deleting
Doubly Linked list:
10
20
30
40
50
Element 30 Deleted successfully
After Deleting
Doubly Linked list:
10
20
40
50
```

Additional Programs

DisplayFiles.java

```
package Java_Lab;

import java.io.File;
public class DisplayFiles
{
        public void printFiles(File[] a, int i, int lvl)
        {
        // base case of the recursion
        // i == a.length means the directory has
        // no more files. Hence, the recursion has to stop
        if(i == a.length)
        {
                return;
        }
        // tabs for providing the indentation
        // for the files of sub-directory
        for (int j = 0; j < lvl; j++)
        {
        System.out.print("\t");
        }
        // checking if the encountered object is a file or not
        if(a[i].isFile())
        {
        System.out.println(a[i].getName());
        }
        // for sub-directories
        else if(a[i].isDirectory())
        {
        System.out.println("[" + a[i].getName() + "]");
        // recursion for sub-directories
        printFiles(a[i].listFiles(), 0, lvl + 1);
        }
        // recursively printing files from the directory
        // i + 1 means look for the next file
        printFiles(a, i + 1, lvl);
        }
        // Main Method
        public static void main(String[] argvs)
        {
                // Providing the full path for the directory
                String path = "D:\\Java Programs";
                // creating a file object
                File f = new File(path);
                // creating on object of the class DisplayFileExample1
                DisplayFiles obj = new DisplayFiles();
                if(f.exists() && f.isDirectory())
                {
                // array for the files of the directory pointed by fObj
                File a[] = f.listFiles();
                // display statements
                System.out.println("= = = =
```

```
    DisplayTableAWT() {
        setTitle("Table Display using AWT");
        setSize(600, 400);
        setLayout(new GridLayout(4, 4, 10, 10));  // dynamic rows/columns with gaps

        try {
            // Open the file
            File file = new File("D:\\Table.txt");
            Scanner sc = new Scanner(file);

            // Read each line and split by commas
            while (sc.hasNextLine()) {
                String line = sc.nextLine();
                String[] items = line.split(a",");

                // Create a Label for each cell
                for (String data : items) {
                    Label lbl = new Label(data.trim());
                    lbl.setBackground(Color.LIGHT_GRAY);
                    add(lbl);
                }
            }
            sc.close();
        } catch (Exception e) {
            add(new Label("Error: " + e.getMessage()));
        }

        setVisible(true);
    }

    public static void main(String[] args) {
        new DisplayTableAWT();
    }
}
```

DisplayTableFromFile.java

```
package Java_Lab;
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.util.*;

public class DisplayTableFromFile extends JFrame {

    public DisplayTableFromFile() {
        setTitle("Table Display from File");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 400);
        setLayout(new BorderLayout());

        // Panel to hold the table content
        JPanel panel = new JPanel();
        panel.setLayout(new GridLayout(0, 1)); // temporary, will reset after knowing column count
        JScrollPane scrollPane = new JScrollPane(panel);

        try {
            // Read the file
            File file = new File("Table.txt");
            Scanner sc = new Scanner(file);

            // Read all lines into a list
            java.util.List<String[]> rows = new ArrayList<>();

            while (sc.hasNextLine()) {
                String line = sc.nextLine();
                String[] columns = line.split(","); // split by comma
                rows.add(columns);
            }
            sc.close();

            // Check if file has content
            if (rows.isEmpty()) {
```

```
                JOptionPane.showMessageDialog(this, "The file is empty!", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            // Determine number of columns from first line
            int cols = rows.get(0).length;

            // Set grid layout with dynamic rows & columns
            panel.setLayout(new GridLayout(rows.size(), cols, 5, 5));

            // Add table elements as labels
            for (int i = 0; i < rows.size(); i++) {
                for (int j = 0; j < cols; j++) {
                    JLabel label = new JLabel(rows.get(i)[j].trim(), SwingConstants.CENTER);
                    label.setOpaque(true);

                    // Highlight header row
                    if (i == 0) {
                        label.setBackground(Color.LIGHT_GRAY);
                        label.setFont(new Font("Arial", Font.BOLD, 14));
                    } else {
                        label.setBackground(Color.WHITE);
                        label.setFont(new Font("Arial", Font.PLAIN, 13));
                    }

                    label.setBorder(BorderFactory.createLineBorder(Color.GRAY));
                    panel.add(label);
                }
            }

        } catch (FileNotFoundException e) {
            JOptionPane.showMessageDialog(this, "File not found!", "Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error reading file: " + e.getMessage(), "Error", JOptionPane.ERRO
        }

        add(scrollPane, BorderLayout.CENTER);
        setVisible(true);
    }

    public static void main(String[] args) {
        new DisplayTableFromFile();
    }
}
```

IntegerDivision.java

```
package Java_Lab;

import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class IntegerDivision extends JFrame implements ActionListener {

    // GUI Components

    JTextField num1Field, num2Field, resultField;

    JButton divideButton;

    JLabel num1Label,num2Label,resultLabel;

    public IntegerDivision() {

        // Frame title

        setTitle("Integer Division");

        setLayout(new GridLayout(4, 2, 10, 10));
```

```
    // Create labels and text fields
    num1Label = new JLabel("Num1:");
    num2Label = new JLabel("Num2:");
    resultLabel = new JLabel("Result:");
    num1Field = new JTextField();
    num2Field = new JTextField();
    resultField = new JTextField();
    resultField.setEditable(false); // result should not be edited
    divideButton = new JButton("Divide");
    divideButton.addActionListener(this);
    // Add components to the frame
    add(num1Label);
    add(num1Field);
    add(num2Label);
    add(num2Field);
    add(resultLabel);
    add(resultField);
    add(new JLabel(""));  // empty label for layout spacing
    add(divideButton);
    // Set frame properties
    setSize(350, 200);
    setVisible(true);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
// Handle button click event
public void actionPerformed(ActionEvent e) {
    try {
        // Read input values
        int num1 = Integer.parseInt(num1Field.getText());
        int num2 = Integer.parseInt(num2Field.getText());
        // Perform division
        int result = num1 / num2;
        // Display result
        resultField.setText(String.valueOf(result));
    } catch (NumberFormatException ex) {
        // If input is not an integer
        JOptionPane.showMessageDialog(this,
```

```
                "Please enter valid integers!",
                "Number Format Error",
                JOptionPane.ERROR_MESSAGE);
        } catch (ArithmeticException ex) {
            // If division by zero occurs
            JOptionPane.showMessageDialog(this,
                    "Cannot divide by zero!",
                    "Arithmetic Error",
                    JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            // Catch-all for any other unexpected exception
            JOptionPane.showMessageDialog(this,
                    "An unexpected error occurred: " + ex.getMessage(),
                    "Error",
                    JOptionPane.ERROR_MESSAGE);
        }
    }
    // Main method
    public static void main(String[] args) {
        new IntegerDivision();
    }
}
```

MouseListenerDemo.java

```
package Java_Lab;
import java.awt.*;
import java.awt.event.*;
class myframe10 extends Frame implements MouseListener
{
    String[] msg=new String[5];

    myframe10()
    {
    this.setSize(500,500);
    this.setVisible(true);
    this.setBackground(Color.blue);
    this.addMouseListener(this);
    }

    public void mouseClicked(MouseEvent e) {
    msg[0]="mouse clicked.................. ("+e.getX()+","+e.getY()+")";
    repaint();
    }

    public void mousePressed(MouseEvent e)
    {
    msg[1]="mouse pressed. ....("+e.getX()+","+e.getY()+")";
    repaint();
    }

    public void mouseReleased(MouseEvent e)
```

```
    {
    msg[2]="mouse released .... ("+e.getX()+","+e.getY()+")";
    repaint();
    }

    public void mouseEntered(MouseEvent e)
    {
    msg[3]="mouse entered. ... ("+e.getX()+","+e.getY()+")";
    repaint();
    }

    public void mouseExited(MouseEvent e)
    {
    msg[4]="mouse exited .... ("+e.getX()+","+e.getY()+")";
    repaint();
    }

    public void paint(Graphics g) {
        int X=50;
        int Y=100;
    for(int i=0;i<msg.length;i++)
    {
     if (msg[i]!=null)
    {
    Y=Y+50;
    g.drawString(msg[i],X,Y);
    }}
    }


}

class MouseListenerDemo
{
public static void main(String[] args)
{
myframe10 f=new myframe10();
}
}
```

MyApplet.java

```
package Java_Lab;
/*
<applet code="MyApplet" width=900 height=500>
</applet>
*/
import java.applet.Applet;
import java.awt.Graphics;

// <applet code="MyApplet.class" width="300" height="200"></applet>

public class MyApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, this is my first Applet!", 50, 100);
    }
}
```

PhoneDirectory.java

```
package Java_Lab;
import java.io.*;
import java.util.*;

public class PhoneDirectory {

   public static void main(String[] args) {
       // Hash tables for quick lookup in both directions
       Hashtable<String, String> nameToPhone = new Hashtable<>();
       Hashtable<String, String> phoneToName = new Hashtable<>();

       try {
           // Open the file
```

```
            BufferedReader br = new BufferedReader(
                    new FileReader("D:\\Java Programs\\Core_Java\\src\\Phone.txt"));
            String line;

            // Read each line
            while ((line = br.readLine()) != null) {
                // Split by tab (\t)
                String[] parts = line.split("\t");
                if (parts.length == 2) {
                    String name = parts[0].trim();
                    String phone = parts[1].trim();

                    // Store in both hash tables
                    nameToPhone.put(name, phone);
                    phoneToName.put(phone, name);
                }
            }
           br.close();


            // Take user input
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter a name or phone number to search: ");
            String input = sc.nextLine().trim();
            // Lookup logic
            if (nameToPhone.containsKey(input)) {
                System.out.println("Phone Number: " + nameToPhone.get(input));
            } else if (phoneToName.containsKey(input)) {
                System.out.println("Name: " + phoneToName.get(input));
            } else {
                System.out.println("No record found for: " + input);
            }
            sc.close();


        } catch (FileNotFoundException e) {
            System.out.println("Error: File 'PhoneDirectory.txt' not found.");
        } catch (IOException e) {
            System.out.println("Error reading from file.");
        }
    }
}
```

SimpleCalculator.java

```
package Java_Lab;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator extends JFrame implements ActionListener {
   JTextField display;
   double num1, num2, result;
   char operator;

   public SimpleCalculator() {
       // Title
       setTitle("Simple Calculator");

       // Text field at top
       display = new JTextField();
       display.setEditable(false);
       add(display, BorderLayout.NORTH);

       // Panel for buttons
       JPanel panel = new JPanel();
       panel.setLayout(new GridLayout(4, 4, 5, 5)); // 4x4 grid

       // Button labels
       String buttons[] = {
           "7", "8", "9", "+",
           "4", "5", "6", "-",
```

```
            "1", "2", "3", "*",
            "0", "%", "=", "C"
        };

        // Create and add buttons
        for (String text : buttons) {
            JButton b = new JButton(text);
            b.addActionListener(this);
            panel.add(b);
        }

        add(panel, BorderLayout.CENTER);

        setSize(300, 300);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        // If a digit is pressed
        if (command.charAt(0) >= '0' && command.charAt(0) <= '9') {
            display.setText(display.getText() + command);
        }
        // If an operator is pressed
        else if (command.equals("+") || command.equals("-") ||
                command.equals("*") || command.equals("%")) {
            try {
                num1 = Double.parseDouble(display.getText());
                operator = command.charAt(0);
                display.setText("");
            } catch (Exception ex) {
                display.setText("Error");
            }
        }
        // If equals (=) is pressed
        else if (command.equals("=")) {
            try {
                num2 = Double.parseDouble(display.getText());
                switch (operator) {
                    case '+': result = num1 + num2; break;
                    case '-': result = num1 - num2; break;
                    case '*': result = num1 * num2; break;
                    case '%':
                        if (num2 == 0)
                            throw new ArithmeticException("Divide by zero");
                        result = num1 % num2;
                        break;
                }
                display.setText(String.valueOf(result));
            } catch (ArithmeticException ex) {
                display.setText("Cannot divide by zero");
            } catch (Exception ex) {
                display.setText("Error");
            }
        }
        // Clear button
        else if (command.equals("C")) {
            display.setText("");
            num1 = num2 = result = 0;
            operator = '\0';
        }
    }

    public static void main(String[] args) {
        new SimpleCalculator();
    }
}
```

TestApplet.java

```
package Java_Lab;
```

```
/*
<applet code="D:\Java Programs\Core_Java\src\Applet1\TestApplet.class"
width=900 height=500>
</applet>
*/
import java.applet.Applet;
import java.awt.Graphics;

class TestApplet extends Applet
{
   String msg=" ";

   public void init()
   {
   msg="init method begins";
   System.out.println(msg);
   }

   public void start()
   {
   msg+="Start method invoked";
   System.out.println(msg);
   }

   public void paint(Graphics g)
   {
   System.out.println(msg);
   msg+="paint method-->WELCOME TO VBIT";
   g.drawString(msg,50,100);
   System.out.println(msg);
   }
}
```

TrafficLightSimulator.java

```
package Java_Lab;

import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class TrafficLightSimulator extends JFrame implements ActionListener {

   // Radio buttons for Red, Yellow, Green

   JRadioButton redButton, yellowButton, greenButton;

   ButtonGroup group;

   // Label to display message

   JLabel messageLabel;

   public TrafficLightSimulator() {

       setTitle("Traffic Light Simulator");

       setSize(400, 300);

       setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

       setLayout(new FlowLayout());

       // Create radio buttons

       redButton = new JRadioButton("Red");

       yellowButton = new JRadioButton("Yellow");

       greenButton = new JRadioButton("Green");
```

```
        // Group buttons (only one can be selected)
        group = new ButtonGroup();
        group.add(redButton);
        group.add(yellowButton);
        group.add(greenButton);
        // Add action listeners
        redButton.addActionListener(this);
        yellowButton.addActionListener(this);
        greenButton.addActionListener(this);
        // Create label for message
        messageLabel = new JLabel(""); // initially no message
        messageLabel.setFont(new Font("Arial", Font.BOLD, 20));
        // Add components to the frame
        add(messageLabel);
        add(redButton);
        add(yellowButton);
        add(greenButton);
        setVisible(true);
    }
  // Handle button clicks
public void actionPerformed(ActionEvent e) {
        if (redButton.isSelected()) {
            messageLabel.setText("STOP");
            messageLabel.setForeground(Color.RED);
        } else if (yellowButton.isSelected()) {
            messageLabel.setText("READY");
            messageLabel.setForeground(Color.ORANGE);
        } else if (greenButton.isSelected()) {
            messageLabel.setText("GO");
            messageLabel.setForeground(Color.GREEN);
        }
    }

    // Main method
    public static void main(String[] args) {
        new TrafficLightSimulator();
    }
```

```
}
```